

How a Kubernetes Cluster Works

Every component and its job, how the cluster boots, the objects it creates, how nodes join, and how Kubernetes decides who you are and what you may do

HOW TO USE THESE NOTES

Read one part per sitting. Each idea goes: **plain meaning** → **analogy** → **our lab** → **interview line**. At the end there's a quiz; answer it on paper *before* looking at the answer key.

Contents

1. The cast: master and worker components
2. Tools vs components: kubectl, crictl, kubeadm, etcdctl
3. One order, followed through every component
4. How the cluster boots (the 4 layers, static pods)
5. What `kubeadm init` does, stage by stage
6. How a worker joins safely (TLS bootstrap)
7. Why nodes are NotReady until the CNI is installed
8. The objects that exist in a fresh cluster
9. Namespaced vs cluster-scoped objects
10. Capacity vs Allocatable, and taints you never added
11. Who are you, and what may you do? (authN, authZ, certificates, ServiceAccounts)
12. Quiz and answer key

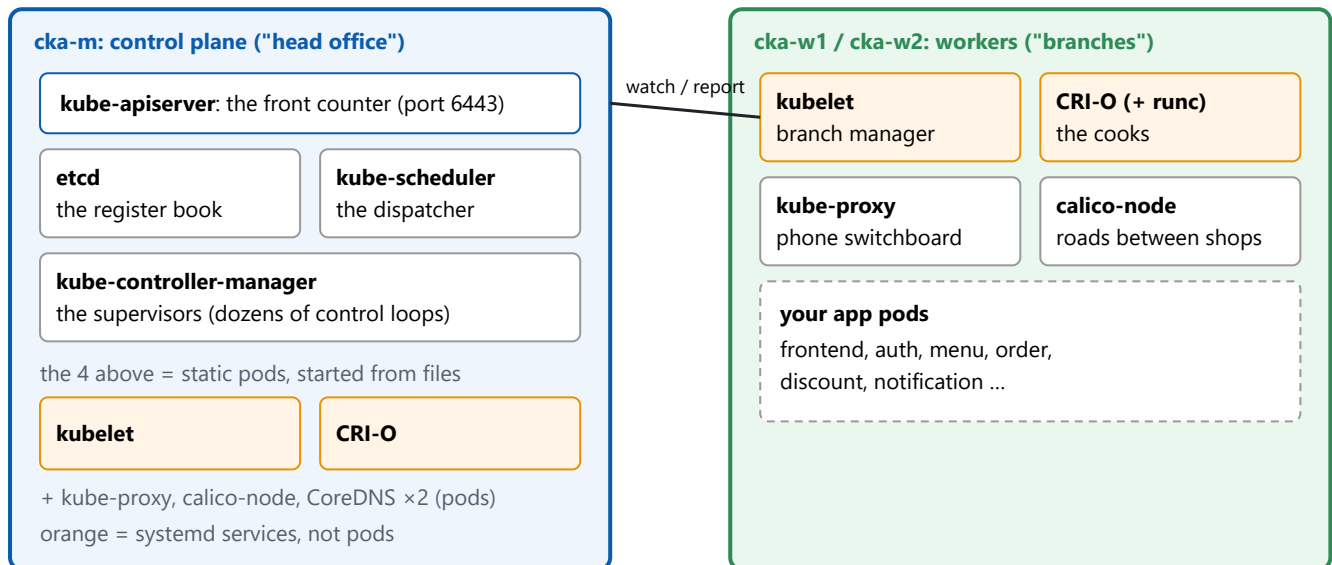
Lab: kubeadm v1.35.8 · CRI-O · Calico v3.32.0 · cka-m (control plane), cka-w1, cka-w2 · written 2026-09-25

1. The cast: master and worker components

The analogy used throughout: GaramChai as a chain of chai shops.

- The **master (control plane)** is the **head office**. It never makes chai itself. It decides, records and supervises.
- The **workers** are the **branches with kitchens**, where the chai (your containers) is actually made.

In our lab, **cka-m** is the head office, and **cka-w1** and **cka-w2** are the branches. The head office also has a small kitchen (kubelet + CRI-O), but a sign on the door, the taint `node-role.kubernetes.io/control-plane:NoSchedule`, says "no customer orders here".



1.1 Control-plane components (on cka-m)

Component	Analogy	Its one job	Runs as
etcd	The register book	Stores every object: Pods, Deployments, Secrets, Nodes. The cluster's only memory	static pod
kube-apiserver	The front counter, the only door	Everyone talks to it, and only it reads or writes etcd. It checks who you are (authentication), what you may do (RBAC), and whether your YAML is valid (admission), then saves it	static pod
kube-scheduler	The dispatcher	Sees a new Pod with no node, picks the best node (enough Allocatable, taints OK, affinity), and writes the decision down . It starts nothing itself	static pod
kube-controller-manager	The supervisors	Dozens of loops: "what's wanted? what exists? fix the difference". Examples: ReplicaSet controller, node lifecycle controller, EndpointSlice controller	static pod

1.2 Node components (on every node, including cka-m)

Component	Analogy	Its one job	Runs as
kubelet	The branch manager	Asks the API server "which pods are assigned to me?", tells CRI-O to run them, runs the probes, reports status. Also starts the static pods on cka-m	systemd service
CRI-O	The cooks	The container runtime : pulls images, creates containers. Under the hood it uses runc (the stove) to start the Linux process	systemd service
kube-proxy	Phone switchboard	Turns Service IPs (e.g. 10.96.0.10) into iptables rules that forward to real pod IPs	DaemonSet pod
Calico (CNI)	Roads between shops	Gives every pod an IP from 172.17.0.0/16 and carries pod traffic between nodes (IPIP tunnels over 10.0.0.x)	DaemonSet pod
CoreDNS	Phone directory	Turns <code>discount-service</code> into its Service IP	Deployment (2 pods)

REMEMBER

Only **kubelet** and **CRI-O** are real systemd services. Something has to exist *before* any pod can run. Everything else is a pod started by the kubelet.

INTERVIEW LINE

"The control plane is the API server, etcd, the scheduler and the controller-manager. The API server is the only component that talks to etcd; everything else watches the API server and reacts. On every node the kubelet runs pods through the container runtime, kube-proxy implements Services, and the CNI plugin provides pod networking."

2. Tools vs components

People mix these up in interviews. A **component** is part of the running cluster; a **tool** is something *you* run to talk to it.

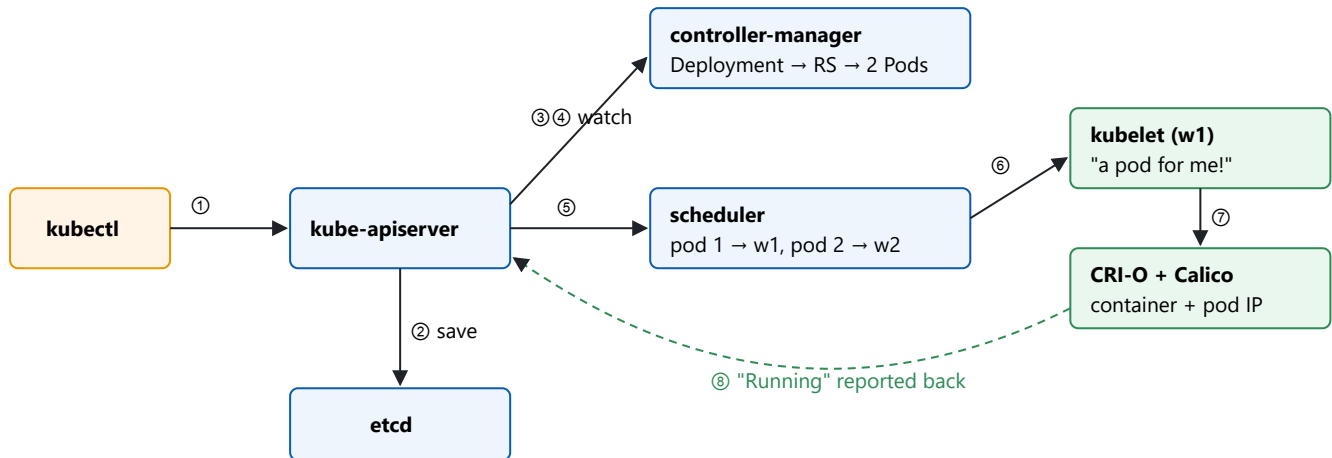
Tool	Talks to	Used for	Analogy
kubectl	the API server , over the network, from anywhere	Everyday work: get, apply, describe, logs	Calling the front counter
crictl	CRI-O on this one node (local socket)	Debugging when the API server is down: <code>crictl ps</code> , <code>crictl logs</code>	Walking into the kitchen and looking at the stove yourself
kubeadm	the node itself	Building the cluster: init, join, upgrade, reset	The construction company
etcdctl / etcdutl	etcd directly	Backup, restore, health	Photocopying the register

KUBECTL VS CRICTL

kubectl shows what Kubernetes *thinks* is running. crictl shows what is *actually* running on that node. When the API server is dead, kubectl is blind and crictl still works. That's why we used `crictl ps` during the etcd restore drill while the control plane was stopped.

3. One order, followed through every component

You run `kubectl create deployment chai --image=nginx --replicas=2`. Here's what happens, in order:



1. **kubectl** sends the Deployment to the **API server**.
2. The API server checks your certificate and RBAC permissions, then saves the Deployment in **etcd**.
3. The **Deployment controller** (inside controller-manager) notices it and creates a **ReplicaSet**.
4. The **ReplicaSet controller** notices "wants 2, has 0" and creates 2 Pod objects with **no node yet**.
5. The **scheduler** picks cka-w1 and cka-w2 and writes that down.
6. The **kubelet on cka-w1** notices "a pod is assigned to me".
7. It asks **CRI-O** to pull and start the container, and CRI-O calls **Calico** for a pod IP.
8. The kubelet reports "Running"; the API server saves it in etcd. `kubectl get pods` shows Running.

THE TWO IDEAS THAT EXPLAIN EVERYTHING

1. **Nobody talks to anybody directly.** Everything goes through the API server.
2. **Every component just watches and reacts.** There's no boss giving orders. If a pod dies, step 4 simply notices "wants 2, has 1" again. That's what "self-healing" means.

4. How the cluster boots: the 4 layers

The puzzle: the API server runs *inside a pod*. But pods are normally created *through* the API server. So how does the first pod ever start?

The answer: static pods. The kubelet reads YAML files from `/etc/kubernetes/manifests/` and starts them **directly from disk**, without asking the API server.

4. Everything else: CoreDNS, then GaramChai's apps

3. Network: calico-node + kube-proxy (DaemonSets) → nodes **Ready**

2. Control plane from files: etcd → API server → controllers, scheduler

1. Linux + systemd start **CRI-O** and **kubelet** (the only non-pods)

#	What comes alive	What it needs	How you'd see it
1	Linux, systemd	nothing	VM console
2	CRI-O and kubelet (systemd services)	the OS	<code>systemctl status kubelet crio</code>
3	kubelet reads <code>/etc/kubernetes/manifests/</code> and starts the 4 static pods	only files on disk	<code>sudo crictl ps</code>
4	etcd loads its data from <code>/var/lib/etcd</code>	disk + PKI	the cluster's memory is back
5	API server connects to etcd, opens 6443	etcd	kubectl answers
6	controller-manager and scheduler connect, elect a leader	API server	<code>kubectl get lease -n kube-system</code>
7	kubelets on all nodes report in (Node status, heartbeat Lease, mirror pods)	API server	<code>kubectl get nodes</code>
8	DaemonSet pods: kube-proxy, calico-node	API server	<code>kubectl get pods -n kube-system -o wide</code>
9	Node Ready (runtime OK + CNI network ready)	CNI	STATUS Ready
10	Normal pods: CoreDNS, then apps	CNI + scheduler	<code>kubectl get pods -A</code>

WHY RESTARTS AFTER A BOOT ARE NORMAL

The 4 static pods start **at the same time**. The API server crashes and retries until etcd answers; the controller-manager and scheduler retry until the API server answers. So a few restarts right after a boot are harmless. Always check **when** a restart happened before worrying.

INTERVIEW LINE

"The kubelet is a systemd service. It starts the control plane as static pods from `/etc/kubernetes/manifests`, which solves the chicken-and-egg problem, because the API server itself runs in a pod. Once the API server is up, the DaemonSets bring up the network, nodes turn Ready, and normal workloads get scheduled. When a cluster won't come up, I debug from the bottom: `systemctl status kubelet` → `crictl ps -a` → `crictl logs` of the API server → `etcd`."

5. What `kubeadm init` does, stage by stage

These are the [tags] you saw in our own init output on 2026-09-23.

Stage	What it does	Linux-boot analogy
[preflight]	Checks swap, ports, runtime, CPU/RAM; pulls images. Stops <i>before</i> touching anything if something is wrong	BIOS self-test
[certs]	Creates the cluster CA and every certificate in <code>/etc/kubernetes/pki</code> (API server, etcd has its own CA, front-proxy, and the <code>sa.key</code> that signs ServiceAccount tokens)	Secure-boot keys
[kubeconfig]	Login files: <code>admin.conf</code> (you), <code>super-admin.conf</code> (break-glass, bypasses RBAC), plus files for kubelet, controller-manager, scheduler	User accounts
[etcd] [control-plane]	Writes the 4 static pod manifests	initramfs on disk
[kubelet-start]	Writes the kubelet config and starts it; it runs the 4 manifests	GRUB loads the kernel
[wait-control-plane]	Waits for health checks. The API server now creates the built-in objects itself: default namespaces, the <code>kubernetes</code> Service, bootstrap RBAC	Kernel up
[upload-config]	Saves the config as ConfigMaps (<code>kubeadm-config</code> , <code>kubelet-config</code>)	/etc saved
[mark-control-plane]	Label + taint <code>control-plane:NoSchedule</code> on the master	—
[bootstrap-token]	Join token (expires in 24 h) + RBAC so new nodes can request certificates	A visitor pass
[addons]	CoreDNS (Deployment + Service <code>kube-dns</code> 10.96.0.10) and kube-proxy (DaemonSet)	Services start

5.1 Our build is a file in Git

We never run `kubeadm init` with loose flags. Everything is in `infra/kubeadm/kubeadm-config.yaml`, so the cluster can be rebuilt identically. The most important lines:

```
kubernetesVersion: v1.35.8           # pinned: a rebuild gives the same cluster
controlPlaneEndpoint: "k8s-api:6443" # a NAME, never a node IP: allows HA later; can't be changed
after init
podSubnet: "172.17.0.0/16"           # pod IPs
serviceSubnet: "10.96.0.0/16"        # Service ClusterIPs
advertiseAddress: "10.0.0.100"        # the API server's IP on the static cluster network
criSocket: unix:///var/run/crio/crio.sock # runtime = CRI-O
evictionHard: memory.available 200Mi, nodefs.available 20%, ... # when the kubelet starts
evicting
```

6. How a worker joins safely (TLS bootstrap)

A new worker has **nothing**: no certificate, no kubeconfig, just the join command. The question is how two strangers learn to trust each other.

Step	What happens	Office analogy
1. Discovery	The worker reads <code>cluster-info</code> (in <code>kube-public</code>). The token proves it came from the real cluster; the CA cert hash in the join command pins the CA, so a fake API server can't pretend	Compare the office's stamp with the fingerprint you were given
2. TLS bootstrap	The kubelet logs in with the token and sends a CSR : "please sign a certificate for <code>system:node:cka-w1</code> "	Use a visitor pass to apply for an employee badge
3. Approval	The CSR is auto-approved and signed by the cluster CA	HR stamps the badge
4. Real credentials	The kubelet saves its certificate; the token is no longer needed	Visitor pass returned
5. Registration	The kubelet creates its Node object (InternalIP = <code>--node-ip</code>)	Name on the office board
6. Heartbeat	Every ~10 s it renews its Lease in <code>kube-node-lease</code> ; missing heartbeats → NotReady	"I'm alive" check-ins
7. DaemonSets	kube-proxy and calico-node are created on the new node automatically	New branch gets phones and roads

WHY WORKERS DON'T GET ADMIN.CONF

A worker's kubelet can only see its own node and pods (the Node authorizer). If an admin kubeconfig sat on a worker, one container escape would give an attacker the whole cluster.

7. Why nodes are NotReady until the CNI is installed

1. Right after `kubeadm init`, there's no network config in `/etc/cni/net.d/`, so the kubelet reports **NetworkReady=false** and the Node is **NotReady**.
2. The node lifecycle controller adds the taint `node.kubernetes.io/not-ready:NoSchedule`.
3. CoreDNS stays **Pending**. In our lab it tolerated `not-ready` only with effect `NoExecute`, not `NoSchedule`. **A toleration must match both the key and the effect.**
4. We install Calico: its DaemonSet copies the CNI binary and config onto each node, gives the node a /26 block of pod IPs, and shares routes over BGP.
5. The config file exists → NetworkReady → node **Ready** → taint removed → CoreDNS gets scheduled.

LINUX ANALOGY

The machine has booted, but the network service hasn't started yet, so nothing that needs the network can come up.

8. The objects that exist in a fresh cluster

Object	Created by	Why it exists
Namespace <code>default</code>	API server	Where objects go if you don't say otherwise. Don't run real apps there
Namespace <code>kube-system</code>	API server	The cluster's own components; can't be deleted
Namespace <code>kube-public</code>	API server	Readable by everyone, even unauthenticated. Holds <code>cluster-info</code> for joining nodes
Namespace <code>kube-node-lease</code>	API server	One Lease per node = cheap, fast heartbeats
Service <code>kubernetes</code> (10.96.0.1)	API server	How pods reach the API server
Service <code>kube-dns</code> (10.96.0.10)	kubeadm	CoreDNS's address, written into every pod's <code>/etc/resolv.conf</code>
ServiceAccount <code>default</code> in every namespace	controller-manager	The identity a pod gets if you don't choose one
ClusterRoles <code>cluster-admin</code> , <code>admin</code> , <code>edit</code> , <code>view</code>	API server	Standard permissions, recreated automatically at every API server start
ClusterRoleBinding <code>kubeadm:cluster-admins</code>	kubeadm	Gives <code>admin.conf</code> its power. Not recreated automatically: losing it caused our 403s during the incident
Leases for controller-manager and scheduler	those components	Leader election: with 3 masters, only one is active at a time
4 static pods	kubelet (from files)	The control plane. Visible via the API as mirror pods ; delete one and it comes straight back, because the file is the truth

9. Namespaced vs cluster-scoped objects

THE COMMON CONFUSION (WE HAD IT TOO)

"kube-system is cluster scope, garamchai is application scope" is **wrong**. Both are just namespaces, the same kind of thing. **Scope is a property of the object type (the kind:), not of a namespace.**

- **Namespaced kind:** every object of this type must live inside *some* namespace (kube-system, garamchai, default...).
- **Cluster-scoped kind:** objects of this type live outside all namespaces. There's no `namespace:` field at all.

The apartment-building analogy

- **Namespaces** = apartments. `kube-system` is the caretaker's flat; `garamchai` is ours. Both are ordinary apartments.
- **Namespaced objects** = furniture. A sofa is always inside some apartment. Two apartments can each have a sofa called "sofa" without any conflict.
- **Cluster-scoped objects** = the building itself: floors, lifts, the list of apartments. They don't belong to one flat, because every flat uses them.

Namespaced (belongs to one app/team)	Cluster-scoped (shared by everyone)
Pod, Deployment, ReplicaSet, StatefulSet, DaemonSet	Node
Service, ConfigMap, Secret, ServiceAccount	Namespace (a flat can't be inside itself)
PersistentVolumeClaim (the app's <i>request</i> for storage)	PersistentVolume (the actual storage), StorageClass
Role, RoleBinding	ClusterRole, ClusterRoleBinding, CustomResourceDefinition, IngressClass

Test to use: "Can two teams each have their own one with the same name?" If yes, it's namespaced. **Rule of thumb:** if the kind's name starts with "Cluster", it's cluster-scoped.

Why is a Node cluster-scoped? It's shared infrastructure: `cka-w1` runs pods from *every* namespace at the same time, so it can't belong to any one of them.

```
kubectl api-resources --namespaced=true      # list namespaced kinds
kubectl api-resources --namespaced=false     # list cluster-scoped kinds
kubectl get nodes -n kube-system              # -n is ignored: nodes have no namespace
```

REAL LESSON FROM PHASE 0

Deleting the `argocd` namespace did **not** delete Argo CD's 3 CRDs and 6 ClusterRoles/Bindings, because they're cluster-scoped. We had to delete them separately.

10. Capacity vs Allocatable, and taints you never added

10.1 Capacity vs Allocatable (per node)

$\text{Allocatable} = \text{Capacity} - \text{kube-reserved} - \text{system-reserved} - \text{eviction threshold}$

Our worker: ~3.8 GiB capacity - (no reservations set) - 200 Mi eviction threshold
≈ 3.58 GiB allocatable

- **Capacity:** everything the node has.
- **Allocatable:** what's left for pods after keeping some back for the OS, the kubelet and a safety margin.
- **The scheduler uses Allocatable, and adds up the pods' requests, not their actual usage.** A node can be 90% idle and still reject a pod, because the requests are "full".

```
kubectl get node cka-w1 -o jsonpath='{.status.capacity.memory}{ " "}{.status.allocatable.memory}{ "\n"}
```

10.2 The disk-pressure taint on cka-w2 (from our audit)

A three-step chain, and interviewers want all three steps:

1. **kubelet** saw free disk fall below its eviction threshold → set the node condition `DiskPressure=True` → **evicted a pod** to free space.
2. **kube-controller-manager** (node lifecycle controller) turned the condition into the taint `node.kubernetes.io/disk-pressure:NoSchedule`.
3. **kube-scheduler** respected the taint and stopped placing pods on cka-w2, so everything crowded onto cka-w1.

It disappeared on its own after we **added a 40 GB disk**: the condition went False (after a ~5-minute wait so it doesn't flap), and the taint was removed.

DON'T REMOVE CONDITION TAINTS BY HAND

Kubernetes owns them and puts them straight back. Fix the cause (the disk), and the taint goes by itself.

Not kube-proxy: kube-proxy only programs Service rules and has nothing to do with disks or taints.

11. Who are you, and what may you do?

11.1 Two questions at every door

Step	Question	Hotel analogy	Failure
Authentication (authN)	Who are you?	Show your ID at reception	401 Unauthorized ("I don't know you"; the name is misleading)
Authorization (authZ)	What may you do?	Your key card only opens your room	403 Forbidden ("I know you, but no")

After both, **admission** runs (adds defaults, enforces policies), then the object is saved. **Our incident:** `kubectl auth whoami` worked (authN OK), but everything returned 403 because the RoleBinding was gone (authZ failed).

11.2 Humans vs programs

The analogy: the API server is a security guard at an office entrance. Humans and delivery robots come through the door, and the guard treats them differently.

	Humans (you, an intern)	Programs (order-service, backup Job)
Identity	A name inside a certificate (<code>CN=</code> = name, <code>O=</code> = group), or a company login (OIDC)	A ServiceAccount object
Stored in the cluster?	No. There's no User object. <code>kubectl get users</code> fails	Yes , in etcd: <code>kubectl get serviceaccounts</code>
How it's checked	The guard checks the card has the CA's seal , then believes the name and group printed on it	A token mounted into the pod
Why	Humans already have an identity elsewhere (HR, AD, SSO)	Programs only exist inside the cluster

```
# See your own ID card (on cka-m):
sudo grep client-certificate-data /etc/kubernetes/admin.conf | awk '{print $2}' | base64 -d \
| openssl x509 -noout -subject -issuer -enddate
# subject = O = kubeadm:cluster-admins, CN = kubernetes-admin    ← group + name
# issuer   = CN = kubernetes                                     ← the cluster CA's seal
```

So **your group comes from the `O=` field of the client certificate inside `admin.conf`**; the file is only the wallet.

11.3 Why the CA key is dangerous

- Whoever holds `ca.key` can print **any** ID card, for example `CN=hacker`, `O=kubeadm:cluster-admins`, and the guard lets them in as admin.
- Kubernetes can't revoke client certificates.** A stolen admin certificate works until it expires (up to 1 year with kubeadm). A stolen password, by contrast, can be changed or disabled in seconds.
- That's why the PKI backup is root-only, `admin.conf` is never shared or committed, and companies log humans in through **OIDC** (single sign-on) instead.

11.4 Least privilege in GaramChai (decided in Phase 1)

Who	Gets
Vishal (admin)	admin.conf, never shared
"intern" user (Phase 8)	Read-only, only in <code>garamchai</code> (certificate via a CSR + Role/RoleBinding)
Each service	Its own ServiceAccount, no API permissions, no token mounted
Backup Job	A small Role in <code>garamchai-data</code> only

12. Quiz

Answer on paper first. The answer key is on the next page.

1. Which is the only component that reads and writes etcd?
a) kubelet · b) kube-scheduler · c) kube-apiserver · d) kube-controller-manager
2. Which two Kubernetes pieces run as systemd services on our nodes, and why can't they be pods?
3. The scheduler has decided a pod goes to cka-w1. Which component actually starts the container?
4. The API server is down. Which tool can still show you the running containers on cka-m?
a) kubectrl · b) crictl · c) kubeadm · d) etcdctl
5. What are static pods, and what problem do they solve?
6. After a reboot, kube-apiserver shows 3 restarts. Is that a problem? What do you check first?
7. A freshly initialised master is NotReady and CoreDNS is Pending. Why? What fixes it?
8. Why must `controlPlaneEndpoint` be a DNS name or VIP, and not a node IP?
9. During a join, what does the `--discovery-token-ca-cert-hash` protect against?
10. Namespaced or cluster-scoped: Pod, Node, Service, Namespace, Secret, PersistentVolume, Deployment, ClusterRole, ConfigMap, StorageClass, Role, PersistentVolumeClaim?
11. Why is a Node cluster-scoped?
12. You deleted a namespace, but some of the app's objects remain. Which ones, and why?
13. A worker has 4 GB RAM but ~3.58 GiB allocatable. Where did the difference go, and which number does the scheduler use?
14. A node is 80% idle, yet a new pod stays Pending with "Insufficient memory". How is that possible?
15. Explain, component by component, how cka-w2 got the `disk-pressure:NoSchedule` taint and how it went away.
16. What's the difference between a 401 and a 403 from the API server?
17. Where does Kubernetes get your group name when you use admin.conf?
18. Why is a stolen admin certificate worse than a stolen password?
19. Why does Kubernetes have a ServiceAccount object but no User object?
20. Which ClusterRoleBinding went missing in our incident, and why didn't it come back by itself when the API server restarted?

Answer key

1. **c) kube-apiserver.** Everything else goes through it.
2. **kubelet and CRI-O.** Something has to exist before any pod can run: the kubelet starts pods, and CRI-O actually runs their containers.
3. The **kubelet** on cka-w1 notices the assignment and asks **CRI-O** to pull the image and start it (Calico gives it an IP).
4. **b) crictl.** It talks to CRI-O on that node directly.
5. Pods defined by files in `/etc/kubernetes/manifests/` that the kubelet runs **without the API server**. They solve the chicken-and-egg problem: the API server runs in a pod, but pods are normally created through the API server.
6. Usually not. All control-plane pods start at once, and the API server retries until etcd is ready. Check **when** the restarts happened (did the count grow after boot?) before worrying.
7. No CNI yet: the kubelet reports `NetworkReady=false`, so the node is `NotReady` and gets the `not-ready:NoSchedule` taint; CoreDNS doesn't tolerate that effect. Installing Calico fixes it.
8. It's baked into certificates and kubeconfigs and is hard to change after init. A name or VIP can later point at a load balancer in front of several masters (HA); a node IP ties the cluster to one machine.
9. A **fake API server**: the new node verifies that the cluster's CA matches the hash it was given, so it won't trust an impostor.
10. Namespaced: Pod, Service, Secret, Deployment, ConfigMap, Role, PersistentVolumeClaim. Cluster-scoped: Node, Namespace, PersistentVolume, ClusterRole, StorageClass.
11. It's shared infrastructure: it runs pods from every namespace at the same time, so it can't belong to any one of them.
12. Cluster-scoped ones: CRDs, ClusterRoles, ClusterRoleBindings (and PVs). Deleting a namespace only deletes objects *inside* it. It happened with Argo CD.
13. Allocatable = capacity minus kube-reserved, system-reserved and the eviction threshold (200 Mi for us), plus memory the kernel keeps. The scheduler uses **Allocatable**.
14. The scheduler adds up **requests**, not real usage. If existing pods requested most of Allocatable, the node is "full" for scheduling even though it's idle.
15. kubelet saw low disk → set `DiskPressure=True` and evicted a pod → node lifecycle controller (controller-manager) added the taint → scheduler avoided the node. After the disk was grown, the condition cleared and the taint was removed automatically.
16. 401 = authentication failed ("who are you?"). 403 = authenticated but not authorised by RBAC ("you may not").
17. From the `o=` **(organisation) field of the client certificate** inside `admin.conf`. The CN is the user name.
18. Client certificates **can't be revoked** in Kubernetes; it stays valid until expiry (up to 1 year). A password can be changed or disabled in seconds, and often has a second factor.
19. Humans are managed outside the cluster (certificates signed by the CA, or OIDC); Kubernetes just trusts the name and groups it's given. Programs live inside the cluster, so Kubernetes manages their identities itself.
20. `kubeadm:cluster-admins`. It's created by kubeadm, not part of the API server's bootstrap RBAC, so the API server doesn't reconcile it; only the built-in ClusterRoles/Bindings come back automatically.